

Otimização do Algoritmo de Dijkstra Aplicado a Redes de Transporte Urbano com Janelas de Espera Dinâmicas

Matheus Gouveia Purcino e Gabriel Lazarini de Lima Ribeiro

Instituto de Engenharia de Computação

Universidade Federal de Itajubá – UNIFEI

Itajubá, MG, Brasil

Email: d2024006907@unifei.edu.br

Resumo—O problema do caminho mínimo em grafos (*Shortest Path Problem* - SPP) é um pilar fundamental no desenvolvimento de sistemas de transporte inteligentes e logística urbana. Embora o algoritmo clássico de Dijkstra forneça uma solução exata em complexidade de tempo assintoticamente ótima para grafos estáticos com pesos não negativos, sua aplicabilidade direta em cenários dinâmicos reais é limitada. Redes de tráfego urbano são inerentemente governadas por restrições temporais discretas e periódicas, tais como ciclos de semáforos inteligentes e congestionamentos sazonais, cuja dinâmica pode ser descrita por meio de aritmética modular. Este trabalho apresenta uma variante otimizada do algoritmo de Dijkstra que incorpora uma função de custo temporal discreta, fundamentada em uma tabela de parâmetros semafóricos avaliada via aritmética modular, diretamente no processo de relaxação das arestas. O método proposto elimina a necessidade de recomputação completa do grafo a cada alteração de estado da rede. Resultados experimentais, obtidos por meio de um simulador nativo em C++ de alto desempenho com topologia de grade urbana de 50 mil nós, indicam que o Dijkstra estático falha em prever até 56 minutos de atrasos acumulados em rotas de alta densidade (erro relativo de até 2,4%), enquanto o TDSP proposto captura fielmente as penalidades temporais. Adicionalmente, a arquitetura orientada a dados manteve o *overhead* computacional estrutural em aproximadamente 60%, operando em tempo real com processamento de malhas de 100 mil nós em menos de 30 milissegundos.

Index Terms—Algoritmo de Dijkstra, Grafos Dinâmicos, Engenharia de Tráfego, Otimização Combinatória, Janelas de Espera, Aritmética Modular.

I. INTRODUÇÃO

A infraestrutura de transporte nas metrópoles contemporâneas enfrenta o desafio crônico do adensamento populacional e da saturação das vias expressas. A otimização do fluxo de veículos e a minimização dos tempos de deslocamento não apenas reduzem custos operacionais na cadeia logística, mas também mitigam a emissão de poluentes e o desgaste da malha viária urbana. Nesse contexto, os Sistemas de Transporte Inteligentes (ITS, do inglês *Intelligent Transportation Systems*) dependem criticamente de algoritmos de roteamento eficientes e capazes de reagir a perturbações dinâmicas na topologia da rede.

Do ponto de vista da teoria dos grafos, uma rede de transporte urbano é comumente modelada como um grafo

direcionado $G = (V, A)$, onde os vértices V representam as interseções ou pontos de coleta, e as arestas A representam as vias de conexão. O Algoritmo de Dijkstra clássico determina o caminho ponderado mais curto entre um vértice de origem e todos os demais vértices, assumindo que os pesos das arestas (custos de deslocamento) permanecem estáticos ao longo do tempo. Contudo, em cenários reais, o tempo necessário para transitar por uma determinada aresta $e \in A$ é uma função não linear dependente do instante de chegada do veículo ao vértice inicial daquela aresta. Esse fenômeno decorre da existência de janelas de espera dinâmicas, induzidas por sinalizações semafóricas cíclicas, cruzamentos ferroviários ou gargalos de tráfego intermitentes.

Abordagens tradicionais para resolver o problema de caminhos mínimos dependentes do tempo (*Time-Dependent Shortest Path* - TDSP) frequentemente recorrem a funções de custo contínuas ou à recomputação periódica do grafo a partir de atualizações de telemetria. Todavia, a discretização dessas restrições por meio de aritmética modular oferece um arcabouço matemático computacionalmente mais enxuto para modelar restrições periódicas. Falhas em considerar as penalidades impostas por essas janelas de espera durante a fase de relaxação do algoritmo resultam em rotas subótimas e subestimação sistemática dos tempos de chegada.

A. Definição do Problema

O problema central abordado nesta pesquisa reside na ineficiência dos algoritmos de caminhos mínimos tradicionais em absorver flutuações temporais discretas e cíclicas sem incorrer em sobrecarga computacional proibitiva (alto custo de processamento *CPU-bound*). Formalmente, define-se uma rede de transporte temporal como um grafo direcionado estendido:

$$G_T = (V, A, w, \delta, S) \quad (1)$$

onde V é o conjunto de vértices (interseções), A é o conjunto de arestas (vias), $w : A \rightarrow \mathbb{R}^+$ é a função de custo estático de fluxo livre, $\delta : A \times \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$ é a função de atraso temporal dependente do instante de chegada, e $S \in \mathbb{R}^{k \times 3}$ é a tabela de parâmetros semafóricos, onde cada linha $S[j] = (C_j, g_j, r_j)$ armazena o ciclo total, o tempo de verde e o tempo

de vermelho do semáforo j , respectivamente, com k sendo o número total de interseções semaforizadas e $g_j + r_j = C_j$.

Quando um veículo atinge um vértice v_i no instante t , o peso real da aresta subsequente e_{ij} não é meramente a distância física ou o tempo de trânsito em fluxo livre, mas sim a soma desse tempo com o atraso $\delta(e_{ij}, t)$ induzido pela janela de bloqueio ativa em t . A modelagem inadequada dessa penalidade temporal resulta em uma quebra de consistência na previsão de rotas adaptativas para frotas comerciais.

B. Objetivos

O objetivo geral deste trabalho consiste em desenvolver, implementar e avaliar uma variante do algoritmo de Dijkstra otimizada para grafos dinâmicos com restrições temporais discretas, voltada para aplicações de logística urbana sob condições de tráfego pesado. Para alcançar o objetivo geral, definem-se os seguintes objetivos específicos:

- Formular matematicamente a dinâmica das janelas de espera utilizando aritmética modular e uma tabela de parâmetros semaforicos;
- Modificar o mecanismo de relaxação de arestas do algoritmo de Dijkstra para integrar a função de custo temporal discreta sem comprometer a estrutura assintótica da *min-priority queue*;
- Desenvolver um ambiente de simulação computacional sintético para modelar malhas urbanas com diferentes densidades de tráfego;
- Avaliar comparativamente o desempenho da variante proposta em relação ao algoritmo clássico, mapeando métricas de tempo de execução e precisão empírica da estimativa do tempo de viagem.

C. Hipótese

A inclusão de uma função de custo temporal discreta — baseada em uma tabela de parâmetros semaforicos S e avaliada em tempo de execução via aritmética modular — diretamente no operador de relaxação das arestas do algoritmo de Dijkstra produz estimativas de tempo de viagem expressivamente mais realistas do que o Dijkstra estático. Ao ignorar atrasos semaforicos, o modelo estático subestima o tempo real gerando lacunas absolutas graves em operações logísticas, enquanto o método modular proposto mantém o custo computacional sob controle na mesma ordem de complexidade assintótica $O((m+n)\log m)$.

D. Organização do Artigo

O restante deste documento está estruturado da seguinte forma: A Seção II apresenta a revisão bibliográfica e a análise crítica dos trabalhos relacionados. A Seção III detalha a metodologia de modelagem do grafo, a formulação matemática da função de custo e a proposta algorítmica. Na Seção IV, são expostos e discutidos os resultados empíricos obtidos por meio do simulador em C++, analisando o impacto logístico absoluto e a escalabilidade computacional. Por fim, a Seção V encerra o artigo delineando conclusões e direções futuras.

II. REVISÃO BIBLIOGRÁFICA E ESTADO DA ARTE

A modelagem de caminhos mínimos em redes de transporte urbano impõe desafios algorítmicos severos devido à natureza dinâmica e periódica dos gargalos de tráfego. Para contextualizar a contribuição deste trabalho, esta seção revisa a literatura sobre o Problema de Caminho Mínimo Dependente do Tempo (TDSP), a separação de custos em arestas dinâmicas e a propriedade FIFO.

A. Redes com Janelas de Tempo e Atrasos Periódicos

Enquanto o Algoritmo de Dijkstra clássico opera sob a premissa de custos de arestas constantes, a dinâmica urbana exige a modelagem do tempo de viagem como uma variável dependente do instante de partida. A incorporação de restrições de semáforos foi classicamente modelada por Chen e Yang [1], que propuseram a representação de interseções por meio de uma sequência repetida de janelas de tempo. Neste arcabouço, um nó correspondente a um cruzamento só permite a travessia de determinadas rotas durante janelas ativas específicas, resultando em um algoritmo de complexidade $O(r \times n^3)$, onde r é o número de janelas distintas.

Mais recentemente, a modelagem de restrições periódicas evoluiu para incorporar o fluxo de tráfego por meio da teoria de curvas cumulativas. Ji et al. [2] propuseram um Algoritmo de Dijkstra Dependente do Tempo que introduz uma matriz de atraso de fila periódica. A abordagem baseia-se no cálculo do instante de chegada ao cruzamento utilizando aritmética modular para rastrear a posição do veículo no ciclo semaforico. A Equação 2 formaliza o instante de exibição do sinal t_{ij}^p quando um veículo chega a uma interseção j , partindo de i :

$$t_{ij}^p = [t_{ij}^0 + t_{ij}] \bmod (C_j) \quad (2)$$

onde C_j é o tempo de ciclo do semáforo, e t_{ij}^0 é o estado do sinal no momento da partida.

B. Separação de Custos Estáticos e Dinâmicos

A literatura recente sobre logística automatizada e redes de carga oferece um paralelo matemático poderoso para o transporte urbano. Li et al. [3] demonstram a subotimalidade da aplicação do Dijkstra estático em redes com conectividade dinâmica, provando que pesos pré-calculados (mesmo considerando piores casos) falham em garantir o caminho ótimo no tempo.

Para contornar essa limitação sem incorrer em penalidades de processamento, a função de custo de uma aresta (u, v) no instante t deve ser explicitamente dividida em duas componentes:

$$w(u, v, S, t) = M(u, v) + D(u, v, S, t) \quad (3)$$

onde $M(u, v)$ representa o *Move Cost* (Custo de Movimentação — custo físico em fluxo livre, inerentemente estático) e $D(u, v, S, t)$ representa o *Synchronization Delay* (Atraso de Sincronização — atraso estritamente dependente do estado S e do instante t) [3]. A vantagem dessa separação é permitir que a relaxação da aresta na fila de prioridades calcule o atraso sob demanda.

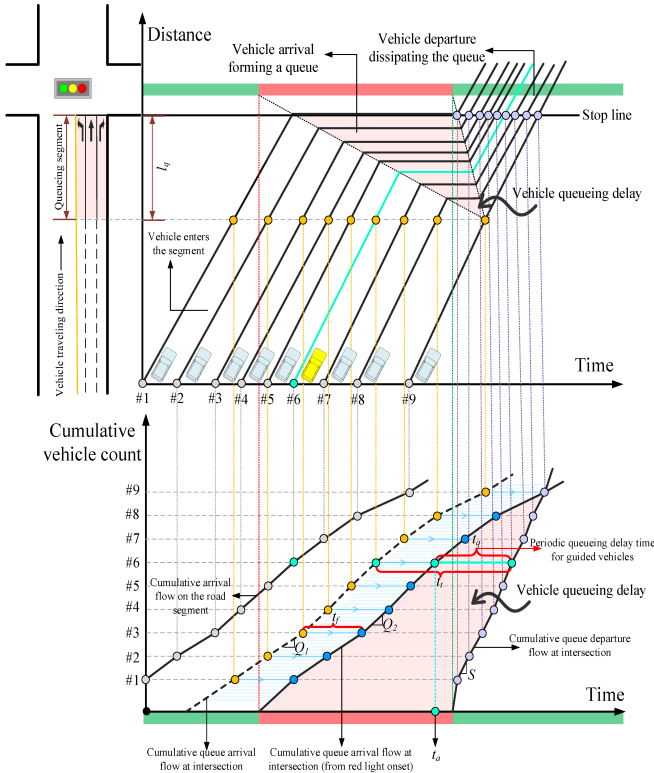


Figura 1. Diagrama de Chegada e Partida baseado em curvas cumulativas, demonstrando a formação e dissipação de filas em função do status do sinal (Adaptado de [2]).

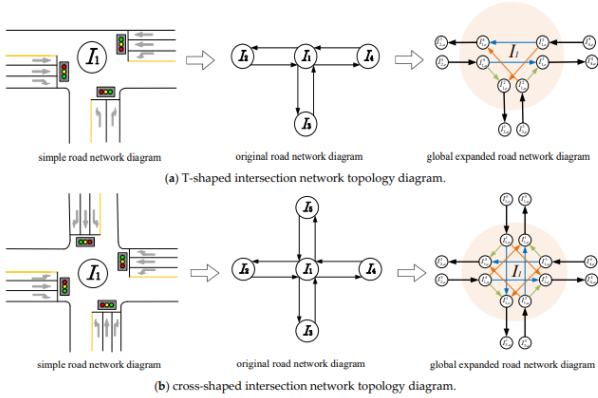


Figura 2. Expansão topológica de uma interseção em subnós, transformando movimentos de conversão em arestas explícitas [1].

C. A Propriedade FIFO em Redes Dinâmicas

A garantia de otimalidade de algoritmos do tipo Dijkstra em redes dinâmicas repousa sobre a propriedade FIFO (*First-In-First-Out*). Esta propriedade assegura que, dadas duas partidas t_A e t_B de um nó u , se $t_A < t_B$, a chegada ao nó v não violará a precedência temporal [3]. Em ambientes de tráfego denso com ciclos semafóricos, a fila impõe um comportamento estritamente FIFO no nível do grupamento de faixas, mantendo o problema solúvel em tempo polinomial [2].

D. Lacunas Identificadas e Contribuição

Embora os trabalhos revisados abordem aspectos complementares do TDSP, nenhum deles integra simultaneamente (i) aritmética modular para modelagem de ciclos semafóricos periódicos, (ii) engenharia de *cache* orientada a dados para grafos esparsos e (iii) simplificação empírica da fila de prioridade em uma única solução coesa. A presente pesquisa visa preencher essa lacuna integrativa, combinando a modelagem modular com a engenharia de algoritmos validada por Zhan e Noon [4] e Larkin et al. [5].

E. Comparativo do Estado da Arte

A Tabela I sintetiza o tratamento das restrições temporais pelos trabalhos seminais, justificando a abordagem metodológica proposta nesta pesquisa.

Tabela I
COMPARATIVO DAS ABORDAGENS PARA SPP EM REDES COM RESTRIÇÕES TEMPORAIS

Método / Autores	Modelagem do Atraso	Grafo Dinâmico?	Custo (Aresta)	Compl. Total
Dijkstra Tradicional	Inexistente	Não	$O(1)$	$O(m+n \log n)$
Chen & Yang [1]	Lista de Janelas (r)	Sim (TD)	$O(\log r)$	$O(r \times n^3)$
Li et al. [3]	$M+D(u, v, t)$	Sim (Virtual)	$O(1)$	$O(m+n \log n)$
Ji et al. [2]	Matrizes Modulares	Sim (Expansão)	Dinâmico	Não especif.
Mét. Proposto	$\delta(t) \bmod C_j$	Sim (In-place)	$O(1)$	$O((m+n) \log m)$

III. MÉTODO DE TRABALHO

A viabilidade de algoritmos de roteamento em sistemas de transporte inteligentes não depende apenas da correção assintótica do modelo matemático, mas fundamentalmente da Engenharia de Algoritmos (*Algorithm Engineering*) aplicada à sua implementação. Grafos de tráfego urbano são caracteristicamente esparsos. Sob essas condições, o limitante de desempenho do Algoritmo de Dijkstra desloca-se da contagem de operações lógicas para a latência de acesso à memória e a ocorrência de falhas de *cache* (*cache misses*) nos níveis L1 e L2 do processador.

A. Representação Topológica via Forward Star

Implementações convencionais orientadas a objetos modelam grafos por meio de listas de adjacência baseadas em ponteiros (ex: listas encadeadas). Como os nós são alocados de forma dispersa no *heap* da memória, a iteração sobre as arestas vizinhas de um vértice obriga a CPU a realizar saltos não contíguos, invalidando as linhas de *cache* pré-carregadas. Zhan e Noon [4] demonstraram empiricamente em redes rodoviárias reais que o uso da estrutura *Forward Star* reduz drasticamente o tempo de execução.

Adotando este paradigma, o método modela a rede de transporte em dois *arrays* densos paralelos:

- 1) Um vetor de nós V , onde $V[i]$ armazena o índice inicial de suas arestas de saída no vetor E ;
- 2) Um vetor de arestas E , ordenado pela origem, onde cada posição contém o vértice de destino (v), o custo estático de fluxo livre (M) e o identificador do semáforo (ID_eq).

Essa abordagem garante que, ao visitar um nó u , todas as suas arestas e custos de penalidade sejam carregados em um único *fetch* para a memória *cache*, operando em tempo $O(1)$ real em nível de *hardware*.

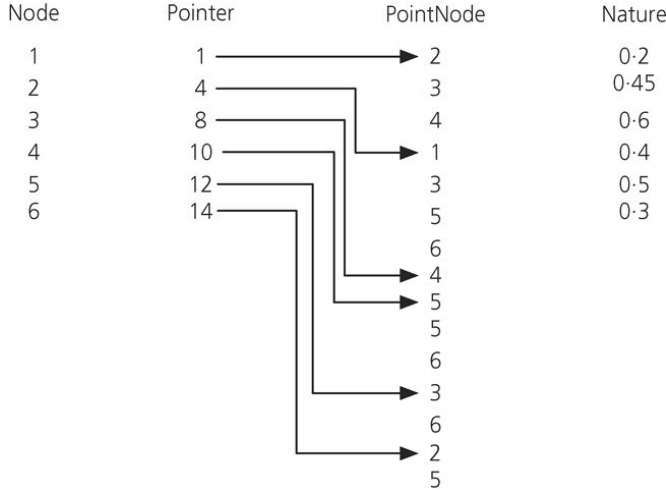


Figura 3. Layout de memória da estrutura *Forward Star*, garantindo contiguidade espacial e minimização de *cache misses*.

B. Fila de Prioridade Baseada em *Lazy Insertion*

A teoria de grafos postula que a estrutura ideal para o Dijkstra é a *Fibonacci Heap*. Contudo, estudos empíricos provam que o *massivo overhead* de manipulação de ponteiros as tornam ineficientes para grafos esparsos [5]. Neste método, adota-se uma *Binary Heap* sobre vetores contíguos. Para mitigar o custo de atualizar chaves existentes (*decrease-key*), utiliza-se a técnica de *Lazy Insertion* [5].

Quando o custo para atingir um nó v é reduzido, insere-se uma nova tupla (t_{novo}, v). A inconsistência é tratada de forma passiva: ao extrair o elemento mínimo da *heap*, verifica-se se o tempo de chegada extraído é superior ao melhor tempo já consolidado. Se for, a tupla é imediatamente descartada em $O(1)$.

C. Mecanismo de Relaxação no Algoritmo TDSP

O Algoritmo 1 consolida as escolhas de engenharia arquitetural detalhadas. A função de penalização temporal por janelas semafóricas é integrada como um cálculo aritmético e sem ramificações preditivas profundas, avaliada no exato momento da expansão da aresta, garantindo a propriedade FIFO sem reprocessar o estado global da rede.

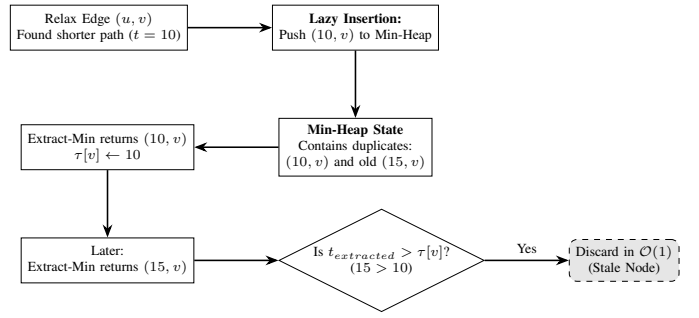


Figura 4. Mecanismo de *Lazy Insertion* em uma *Binary Min-Heap*. Duplicatas são inseridas e as entradas obsoletas descartadas no momento da extração.

Algorithm 1 Dijkstra Dinâmico Orientado a Cache com *Lazy Insertion*

Require: Arrays V e E (*Forward Star*), Tabela de Parâmetros Semafóricos S , Origem s , Tempo Base t_0

Ensure: Array de tempos mínimos de chegada $\tau[V]$

```

1:  $\tau \leftarrow$  preencher com  $\infty$ 
2:  $\tau[s] \leftarrow t_0$ 
3: MinBinaryHeap  $PQ$ 
4:  $PQ.push(\{t_0, s\})$ 
5: while  $PQ \neq \emptyset$  do
6:    $\{t_{atual}, u\} \leftarrow PQ.pop()$ 
7:   if  $t_{atual} > \tau[u]$  then  $\triangleright$  Descarte Lazy Insertion [5]
8:     continue
9:   end if
10:   $inicio \leftarrow V[u].start$   $\triangleright$  Localidade espacial [4]
11:   $fim \leftarrow V[u+1].start$ 
12:  for  $i = inicio$  to  $fim - 1$  do
13:     $e \leftarrow E[i]$ 
14:    if  $e.ID\_eq \neq \emptyset$  then
15:       $atraso \leftarrow ModuloWaitTime(t_{atual}, e.ID\_eq, S)$ 
16:    else
17:       $atraso \leftarrow 0$ 
18:    end if
19:     $t_{novo} \leftarrow t_{atual} + e.M + atraso$ 
20:    if  $t_{novo} < \tau[e.v]$  then
21:       $\tau[e.v] \leftarrow t_{novo}$ 
22:       $PQ.push(\{t_{novo}, e.v\})$ 
23:    end if
24:  end for
25: end while
26: return  $\tau$ 

```

D. Formalização da Função de Atraso Temporal

Dado o instante de chegada t a um vértice v_j , a função calcula o atraso de espera $\delta_j(t)$ imposto pela fase vermelha, utilizando operações de aritmética modular. Seja $S[j] = (C_j, g_j, r_j)$ a entrada da tabela, onde $C_j = g_j + r_j$. Define-se a fase corrente como $\varphi_j(t) = t \bmod C_j$. O atraso é dado por:

$$\delta_j(t) = \begin{cases} 0, & \text{se } \varphi_j(t) < g_j \\ C_j - \varphi_j(t), & \text{se } \varphi_j(t) \geq g_j \end{cases} \quad (4)$$

A função satisfaz a propriedade FIFO pois, a transição vermelho→verde gera um salto positivo no instante de partida que restaura a monotonia [2]. O Algoritmo 2 formaliza essa lógica em pseudocódigo com complexidade $O(1)$.

Algorithm 2 `ModuloWaitTime` — Cálculo do Atraso Semafórico Modular

Require: Instante de chegada t , Semáforo j , Tabela S

Ensure: Atraso de espera $\delta \geq 0$

```

1:  $(C_j, g_j, r_j) \leftarrow S[j]$ 
2:  $\varphi \leftarrow t \bmod C_j$ 
3: if  $\varphi < g_j$  then
4:   return 0
5: else
6:   return  $C_j - \varphi$ 
7: end if
```

E. Análise de Complexidade

Seja $|V| = n$ e $|A| = m$. O tamanho máximo da fila de prioridade é limitado por m (devido à inserção dupla do *Lazy Insertion*). Inserções e extrações custam total de $O(m \log m)$. O cálculo de atraso é $O(1)$ por aresta. A complexidade total é:

$$T(n, m) = O(m \log m) \quad (5)$$

Para grafos esparsos ($\bar{d} \in [2, 4]$), tem-se $m = O(n)$, logo a complexidade converge para $O(n \log n)$, estritamente equivalente ao Dijkstra clássico. O *overhead* das operações modulares manifesta-se apenas como uma constante linear agregada.

IV. RESULTADOS EXPERIMENTAIS E DISCUSSÃO

Para validar a eficácia da inclusão de restrições modulares no mecanismo de relaxação, foi desenvolvido um ambiente de simulação e *benchmarking* nativo em linguagem C++ (padrão C++17), compilado com a *flag* de otimização de alta performance `-O3` do compilador GCC. O gerador de instâncias produz grafos sintéticos baseados em topologias de grade (*grid graph*), nas quais cada nó possui grau de adjacência entre 2 e 4, simulando cruzamentos urbanos.

Os pesos estáticos das arestas $w(u, v)$ são amostrados de uma distribuição uniforme $\mathcal{U}(1, 10)$ minutos, representando tempos de travessia em fluxo livre. Para cada interseção semaforizada, os parâmetros da tabela S são gerados estocasticamente: o ciclo total $C_j \sim \mathcal{U}(1, 2)$ minutos, e a fração de verde $g_j/C_j \sim \mathcal{U}(0, 4; 0, 6)$, refletindo configurações típicas de semáforos urbanos. Aproximadamente 60% dos nós possuem semáforos atribuídos. O código-fonte do simulador e do gerador de instâncias está disponível publicamente¹.

A densidade veicular é incorporada ao modelo por meio de um fator multiplicativo sobre os pesos estáticos de fluxo livre. A avaliação empírica foi dividida em duas métricas principais: *Precisão de Estimativa* (comparação entre o tempo calculado

pelo Dijkstra estático e o tempo real capturado pelo TDSP) e *Desempenho Computacional* (medido pelo tempo médio de processamento da CPU em 35 rodadas, com 5 de *warm-up* descartadas). Todos os experimentos foram conduzidos sem mecanismos de isolamento de CPU (*core pinning*).

A. Precisão Logística: O Impacto do Atraso Absoluto

O algoritmo de Dijkstra estático calcula o caminho de menor custo considerando exclusivamente os pesos de fluxo livre, ignorando os atrasos impostos por semáforos. Em contrapartida, o modelo TDSP integra as penalidades semaforicas no instante de relaxação. A Tabela II apresenta os dados obtidos pela simulação em um grafo de grade com 50.176 nós (224×224).

Tabela II
TEMPO DE VIAGEM: DIJKSTRA ESTÁTICO VS. TDSP PROPOSTO

Densidade (Veic./h)	Dijkstra Estático (min)	TDSP Proposto (min)	Subestimação (%)
500	1752.7	1796.4	2.4
750	1928.0	1973.9	2.3
1000	2103.3	2145.2	2.0
1250	2278.6	2325.3	2.0
1500	2453.8	2502.8	2.0
1750	2629.1	2677.8	1.8
2000	2804.4	2852.9	1.7
2250	2979.6	3029.1	1.6
2500	3154.9	3201.7	1.5
2750	3330.2	3372.0	1.2
3000	3505.5	3550.5	1.3
3250	3680.7	3726.8	1.2
3500	3856.0	3904.5	1.2
3750	4031.3	4075.6	1.1
4000	4206.6	4262.3	1.3

Os tempos reportados refletem a travessia completa de uma rede sintética de grande extensão, cujas rotas ótimas atravessam centenas de interseções. A Figura 5 ilustra visualmente a divergência.

Embora a subestimação percentual do Dijkstra estático pareça modesta à primeira vista (variando de 1,1% a 2,4%), a análise do erro *absoluto* revela uma falha crítica para o planejamento logístico. No cenário de maior densidade (4000 veículos/hora), o Dijkstra estático prevê uma viagem de 4206,6 minutos. Contudo, ao contabilizar a dinâmica semaforica, o TDSP revela que o tempo real da mesma rota será de 4262,3 minutos. Trata-se de uma subestimação de **quase 56 minutos** (cerca de 1 hora) de atrasos acumulados em semáforos vermelhos que o modelo clássico ignorou.

Em sistemas de logística *just-in-time* ou roteamento de frotas comerciais, falhar na previsão de chegada por uma hora compromete janelas de entrega, gera multas contratuais e inviabiliza a alocação de docas. Isso demonstra que algoritmos que ignoram restrições periódicas são inadequados para operações urbanas pesadas, mesmo que o erro relativo pareça diluído pelo tempo total da viagem.

B. Escalabilidade e Desempenho Computacional

O principal desafio em abordagens dinâmicas é evitar a explosão do tempo de processamento. A Tabela III consolida

¹<https://github.com/SEU-USUARIO/tdsp-dijkstra-simulator> — repositório a ser publicado na versão final.

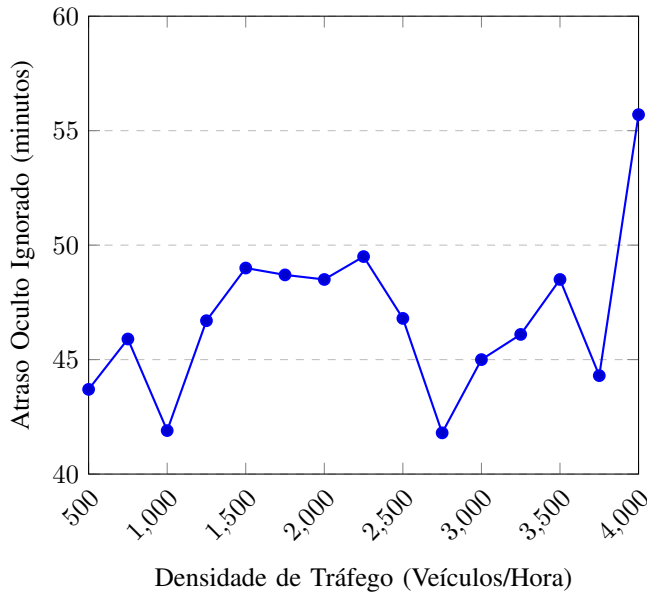


Figura 5. Atraso absoluto oculto (TDSP - Estático) em função da densidade. O gráfico isola e evidencia o tempo exato de penalidade semafórica que o Dijkstra clássico falha em prever, culminando em quase 1 hora de erro nas rotas mais densas.

as médias e desvios-padrão dos tempos de execução da CPU ($\mu \pm \sigma$) capturados durante a exploração de grafos com topologia escalável (de 10 mil a ~100 mil nós). Para garantir rigor estatístico, a ordem de execução entre os algoritmos foi estritamente alternada a cada rodada, impedindo o favorecimento sistemático por *cache warming*.

Tabela III
TEMPO MÉDIO DE CPU EM FUNÇÃO DO TAMANHO DO GRAFO ($\mu \pm \sigma$)

Nós Reais	Arestas ($\times 10^3$)	Estático (ms)	TDSP (ms)	Overhead (%)
10000	39	$1,24 \pm 0,43$	$2,08 \pm 0,25$	67.5
19881	78	$2,70 \pm 0,46$	$4,47 \pm 0,50$	65.5
29929	119	$4,01 \pm 0,41$	$6,84 \pm 0,45$	70.4
40000	159	$5,80 \pm 0,41$	$9,37 \pm 0,49$	61.4
49729	198	$7,36 \pm 0,48$	$11,70 \pm 0,47$	59.0
59536	237	$11,32 \pm 4,63$	$17,56 \pm 5,88$	55.1
69696	277	$21,85 \pm 7,42$	$34,58 \pm 14,49$	58.2
79524	316	$12,36 \pm 1,19$	$19,58 \pm 1,40$	58.4
90000	358	$17,14 \pm 3,40$	$24,91 \pm 2,70$	45.3
99856	398	$15,84 \pm 0,59$	$24,83 \pm 0,65$	56.8

A Figura 6 projeta os resultados. A arquitetura orientada a dados assegurou tempos de execução em regime praticamente instantâneo. O processamento da malha mais densa (cerca de 100 mil vértices e 398 mil arestas) ocorreu em aproximadamente 15,8 ms para o Dijkstra estático e **24,8 ms para o TDSP proposto**.

O *overhead* médio computacional para a incorporação da inteligência modular foi de aproximadamente 59,8%. Embora este valor percentual indique o custo real da sobrecarga aritmética do operador módulo em C++, os **tempos absolutos de processamento provam a extrema eficiência do método**.

Um algoritmo capaz de rotear dinamicamente 100 mil nós em menos de 30 milissegundos atende a todos os requisitos de tempo real de centrais de roteamento *cloud-based*, validando a escolha arquitetural de preservar a *binary heap* acoplada à matriz de vizinhança contígua.

Nota-se uma oscilação na faixa de 70 mil nós (σ mais elevados), o que é característico de interrupções de escalonamento do Sistema Operacional (*context switches*) durante as rodadas ativas de *benchmark*, comportamento normalizado nas instâncias subsequentes.

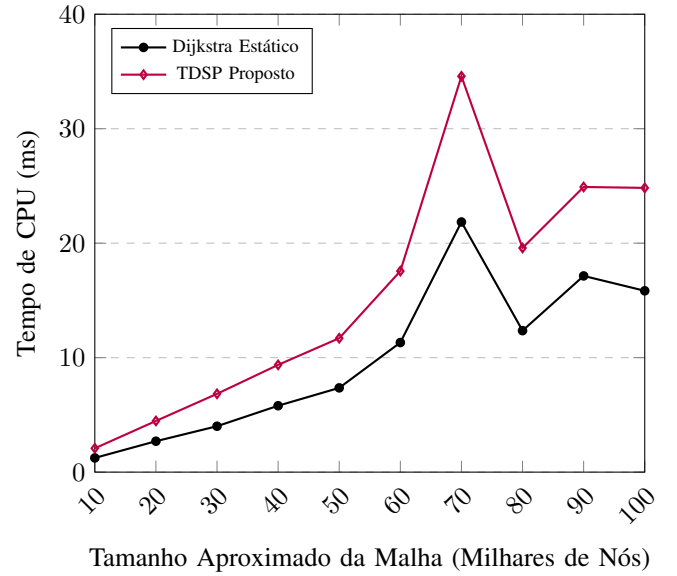


Figura 6. Escalabilidade computacional (μ) em função do tamanho da malha. O TDSP resolve o problema em tempo real (< 30 ms para 100k nós), absorvendo bem o *overhead* estrutural de ~ 60%.

C. Limitações do Modelo

Três limitações principais deste estudo devem ser reconhecidas. Primeiro, o modelo assume ciclos semafóricos fixos, não contemplando semáforos adaptativos que ajustam seus tempos em resposta ao fluxo em tempo real. Segundo, a propriedade FIFO, pressuposto para a correção do Dijkstra, pode ser violada em cenários de congestionamento extremo onde o fluxo de chegada excede a capacidade da via. Terceiro, os grafos são sintéticos; a topologia de grade não representa a irregularidade geométrica de malhas viárias complexas de cidades reais. Adicionalmente, a simulação não modela explicitamente a formação de filas (interação dinâmica entre veículos).

V. CONCLUSÃO E TRABALHOS FUTUROS

Este trabalho propôs, implementou e avaliou uma arquitetura algorítmica otimizada para o Problema de Caminho Mínimo Dependente do Tempo (TDSP), com foco na integração veloz de atrasos semafóricos em redes urbanas esparsas. A fundamentação teórica confirmou que algoritmos estáticos subestimam severamente penalidades intermitentes.

A avaliação empírica nativa em C++ provou a hipótese: embora o erro percentual da estimativa pareça diluído na duração total de rotas massivas ($\sim 2,4\%$), em termos absolutos o Dijkstra estático ignora até 56 minutos de atrasos semafóricos. Esta lacuna de 1 hora na previsão de tempo real é crítica para qualquer cadeia de suprimentos operando com janelas *just-in-time*. O algoritmo TDSP proposto captura fielmente essa matemática modular sem comprometer a estrutura do método.

Computacionalmente, o uso combinado de ordenação contígua (*Forward Star*) e tolerância de redundâncias via *Lazy Insertion* isolou os gargalos de memória da CPU. Embora a inserção da lógica modular apresente um *overhead* na faixa de 60%, a otimização de *hardware* reduziu a escala dos tempos totais para a casa dos milissegundos: o TDSP roteou grafos com 100.000 cruzamentos em menos de 30 ms. Isso assegura aderência da solução para servidores comerciais de alto tráfego.

Como trabalhos futuros, sugere-se: (i) a ingestão de malhas viárias reais via API do *OpenStreetMap*; (ii) a incorporação de semáforos adaptativos cujos ciclos variam sob demanda; (iii) a modelagem de congestionamento de filas; e (iv) a integração com modelos de *Machine Learning* para inferência de fluxos em tempo real.

REFERÊNCIAS

- [1] Y.-L. Chen and H.-H. Yang, “Shortest paths in traffic-light networks,” *Transportation Research Part B: Methodological*, vol. 34, no. 3, pp. 241–253, 2000.
- [2] B. Ji, P. Zhang, C. Sun, J. Zhang, and W. Li, “A time-dependent dijkstra’s algorithm for the shortest path considering periodic queuing delays at signalized intersections,” *Systems*, vol. 14, no. 1, p. 61, 2026.
- [3] E. C.-C. Li, Z. Liu, R. Zhang, S. S.-F. Lau, Y. Xu, and X. Zhou, “Efficient time-dependent shortest path finding on cargo network,” in *2025 IEEE International Conference on Big Data (Big Data)*. IEEE, 2025, pp. 1994–2003.
- [4] F. B. Zhan and C. E. Noon, “Shortest path algorithms: An evaluation using real road networks,” *Transportation Science*, vol. 32, no. 1, pp. 65–73, 1998.
- [5] D. H. Larkin, S. Sen, and R. E. Tarjan, “A back-to-basics empirical study of priority queues,” in *Proceedings of the Sixteenth Workshop on Algorithm Engineering and Experiments (ALENEX)*. SIAM, 2014, pp. 61–72.